

V0088

Advanced Information Security

高级信息安全

Instructor: **Prof. Xiaohong Yu**

Graduate School of Information and Communication Technology,
Ajou University, Korea

高级密码分析

- 现代分组密码分析
 - 差分分析
 - 线性分析
- 对 RSA 的侧信道攻击
- 对背包密码的格约简攻击
- 对DES的Hellman 时间-存储权衡攻击

分组密码分析技术

- 线性分析和差分分析都是为了攻击DES而开发的。
 - 也适用于其他分组密码
- 差分分析： Biham和 Shamir 在1990提出
 - 20世纪70年代参与设计DES的人掌握了差分分析。
 - 用于分析分组密码的弱点，而非实际攻击
 - 选择明文攻击
- 线性分析——matsui 在1993提出
 - DES没有针对线性分析进行设计优化
 - 比差分分析略接近实际攻击的实现。
 - 已知的明文攻击

DES分组密码

• DES:

- 16轮 Feistel 结构密码
- 分组长度: 64位
- 密钥: 56位
- DES每轮: 使用48位子密钥

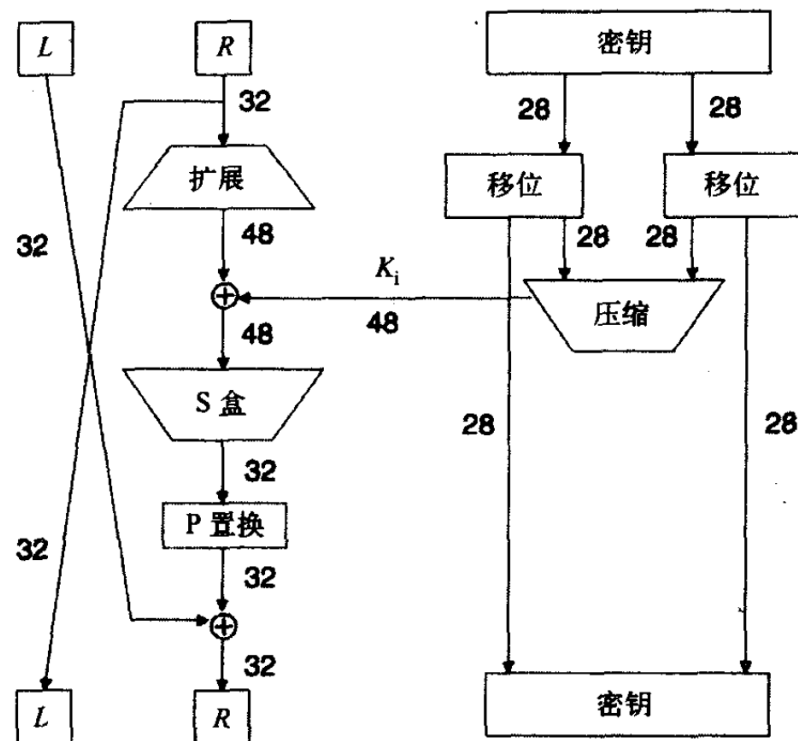
$$P = (L_0, R_0)$$

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

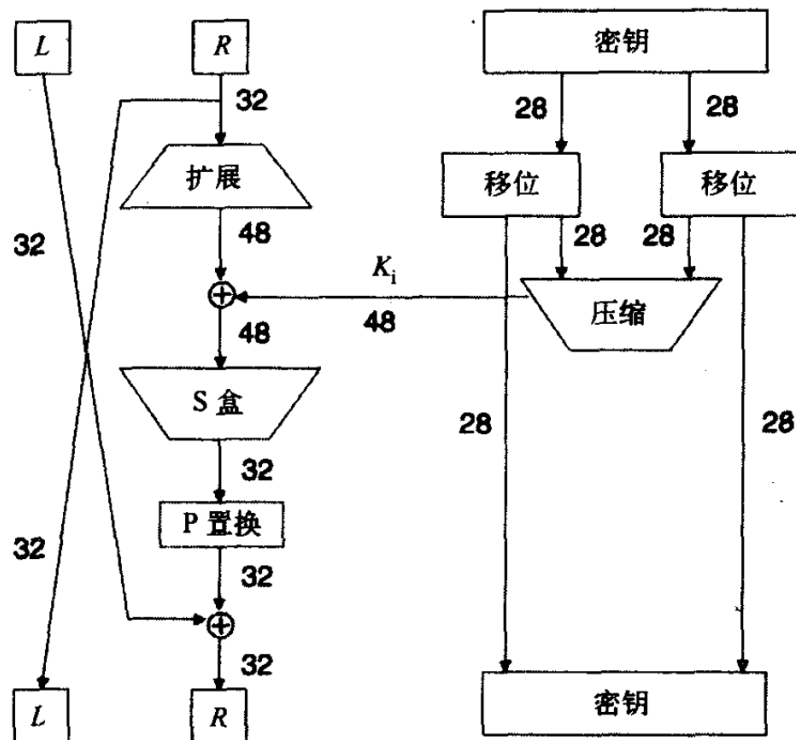
$$F(R_{i-1}, K_i) = \text{P-box}(\text{S-boxes}(\text{Expand}(R_{i-1}) \oplus K_i))$$

(P-box: P 置换; S-boxes: S 盒; Expand: 扩展)



DES分组密码

- S盒: 48位缩减到32位
- S盒数量: 8个S盒,
- S盒操作: 6位映射出4位



b_0b_5	$b_1b_2b_3b_4$															
00	1110	0100	1101	0001	0010	1111	1011	1000	0011	1010	0110	1100	0101	1001	0000	0111
01	0000	1111	0111	0100	1110	0010	1101	0001	1010	0110	1100	1011	1001	0101	0011	1000
10	0100	0001	1110	1000	1101	0110	0010	1011	1111	1100	1001	0111	0011	1010	0101	0000
11	1111	1100	1000	0010	0100	1001	0001	0111	0101	1011	0011	1110	1010	0000	0110	1101

差分密码分析

- 除了S盒之外，DES的所有部分都是线性的。
- 差分分析旨在克服这种非线性。
- 目的是比较输入和输出的**差异**。
- 为简单起见，首先只**考虑一轮和一个S盒**。

差分密码分析

- 比较输入差分和输出差分
- 简化的S盒：3进2出
- 输入： $x_0x_1x_2$
- S盒操作： $Sbox(010)=11$

		列 x_1x_2			
		00	01	10	11
x_0	0	10	01	11	00
	1	00	10	01	11

差分密码分析

- 输入:

- $X_1 = 110, X_2 = 010$

- 密钥:

- $K = 011$

		列 x_1x_2			
		00	01	10	11
x_0	0	10	01	11	00
	1	00	10	01	11

$$Sbox(X_1 \oplus K) = 10$$

$$Sbox(X_2 \oplus K) = 01$$

- 假设K未知, 根据表格可知:

- $X_1 \oplus K = \{000, 101\}, X_2 \oplus K = \{001, 110\}$

- $\Rightarrow K \in \{110, 011\} \cap \{011, 100\} = 011$

差分密码分析

- 攻击一个S盒没什么用!
 - 而且，攻击者并非总能看到输入和输出。
- 为了实现这一点，我们必须做两件事。
 1. 将之前的分析扩展到整个一轮的分析
 - 同时考虑DES的8个S盒
 - 选择输入源，使只有一个 S 盒“active”。
 2. 然后扩大分析到**所有轮**
 - 上一轮的输出作为下一轮的输入。
 - 选择合适的输入，使输出对下一轮“良好”。

差分密码分析

- 我们处理输入和输出差异
- 假设我们知道输入 X_1 和 X_2
 - 对于 X_1 , S 盒的输入是 $X_1 \oplus K$
 - 为了 X_2 , S 盒的输入是 $X_2 \oplus K$
 - 密钥K未知
 - 输入差值: $(X_1 \oplus K) \oplus (X_2 \oplus K) = X \oplus X$
- 输入差值与K无关
- 输出差值: $Y_1 \oplus Y_2$ 几乎是下一轮的输入差异
- 目标是在轮函数迭代中“串联”差分。

差分密码分析

- 如果我们能从已知的输入差值中获得已知的输出差值.....
 - 或许能够通过轮函数迭代来链接起这些差分
 - 如果这种情况只以一定的概率发生，那也没关系。
- 如果输入差异为 0...
 - ...输出差异一定为0，即输入相同时，输出也一定相同。
 - 通过选择输入差分为0，使得某些 S 盒“不活动”。

差分密码分析

- 找所有输入值 X , X_1, X_2 : $X = X_1 \oplus X_2$
- 计算对应的输出差分: $Y = Y_1 \oplus Y_2 = Sbox(X_1) \oplus Sbox(X_2)$.
- 统计结果:

表 6.1 S 盒差分分析

$X_1 \oplus X_2$	$Sbox(X_1) \oplus Sbox(X_2)$			
	00	01	10	11
000	8	0	0	0
001	0	0	4	4
010	0	8	0	0
011	0	0	4	4
100	0	0	4	4
101	4	4	0	0
110	0	0	4	4
111	4	4	0	0

差分密码分析

- 输入差分为000时，输出差分也总是00.
- 输入差分是010时，输出差分总是01,
- 这里 $X_1 \oplus X_2 = 010$ 输出偏差一致，可以用于DES多轮差分分析

表 6.1 S 盒差分分析

$X_1 \oplus X_2$	$sbox(X_1) \oplus sbox(X_2)$			
	00	01	10	11
000	8	0	0	0
001	0	0	4	4
010	0	8	0	0
011	0	0	4	4
100	0	0	4	4
101	4	4	0	0
110	0	0	4	4
111	4	4	0	0

线性密码分析

- 与差分分析类似，也是分析分组密码的**非线性部分**。
- 目标：找出与密码非线性部件近似等价的**线性方程**。
- 对于类似DES的密码，我们需要用线性函数来近似S盒。

$$F(R_{i-1}, K_i) = \text{P-box}(\text{S-boxes}(\text{Expand}(R_{i-1}) \oplus K_i))$$

(P-box: P 置换; S-boxes: S 盒; Expand: 扩展)

线性密码分析

- 输入: $x_0x_1x_2$, 输出: y_0y_1
- 统计各种可能线性逼近数目统计。
- 输入 $x_0x_1x_2$ 共有 2^3 个输出值。

行	列 x_1x_2			
	00	01	10	11
x_0 0	10	01	11	00
1	00	10	01	11

可以通过线性
方程把S盒替
换

输入位		输出位		
		y_0	y_1	$y_0 \oplus y_1$
0		4	4	4
x_0		4	4	4
x_1		4	6	2
x_2		4	4	4
$x_0 \oplus x_1$		4	2	2
$x_0 \oplus x_2$	$y_0 = x_0 \oplus x_2 \oplus 1$	0	4	4
$x_1 \oplus x_2$		4	6	6
$x_0 \oplus x_1 \oplus x_2$		4	6	2

$y_1 = x_1$: 概率为3/4

概率为1

$y_0 \oplus y_1 = x_1 \oplus x_2$:

概率为3/4

线性密码分析

- 考虑DES的一轮的S盒
- 令 $Y = \text{Sbox}(X)$
- 假设 $y_3 = x_2 \oplus x_5$ 有很高的概率
 - 即，输出 y_3 具有良好的线性近似
- 我们能否扩展这个方法，从而求解线性方程组得到密钥？
- 与差分密码分析一样，我们需要“串联”多轮运算。

对**DES**进行线性密码分析

- 除了S盒之外，DES是线性的。
- 我们能用线性函数多好地逼近 S 盒？
- DES S盒的设计使得任何一个输出位都无法用良好的线性近似来表示。
- 但是，输出位的线性**组合**可以用输入位的线性组合来近似表示。

微型DES (TDES)

- TDES:

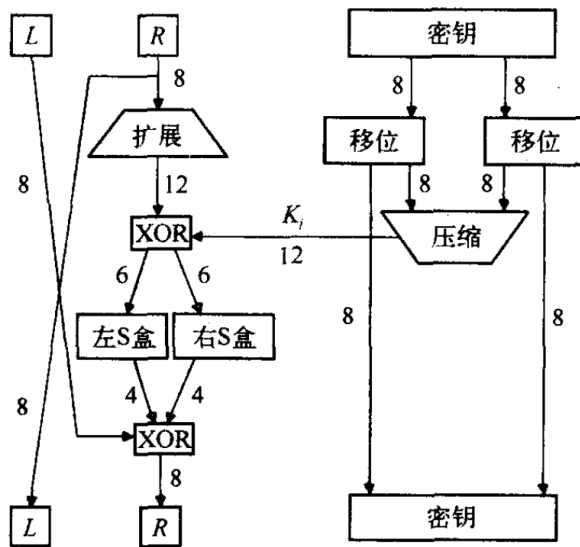
- 16 位分组长度
- 16 位密钥
- 4轮
- 2 个 S 盒, 6进4出
- 每轮使用12位子密钥
- 纯文本 = (L_0, R_0)
- 密文 = (L_4, R_4)

- DES:

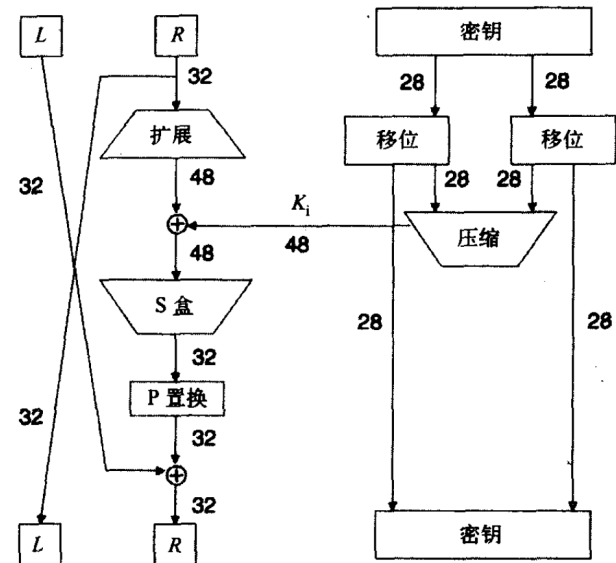
- 分组长度: 64位
- 密钥: 56位
- 16轮
- 8个S盒, 6进4出
- 每轮使用48位子密钥
- 纯文本 = (L_0, R_0)
- 密文 = (L_{16}, R_{16})

微型DES (TDES)

- 与DES相比，TDES去掉了P置换，初始置换和逆初始置换。
- TDES由于短的密钥和分组长度不能提供任何实际的安全。
- 只是作为理解线性分析和差分分析的工具。



TDES



DES

微型DES (TDES)

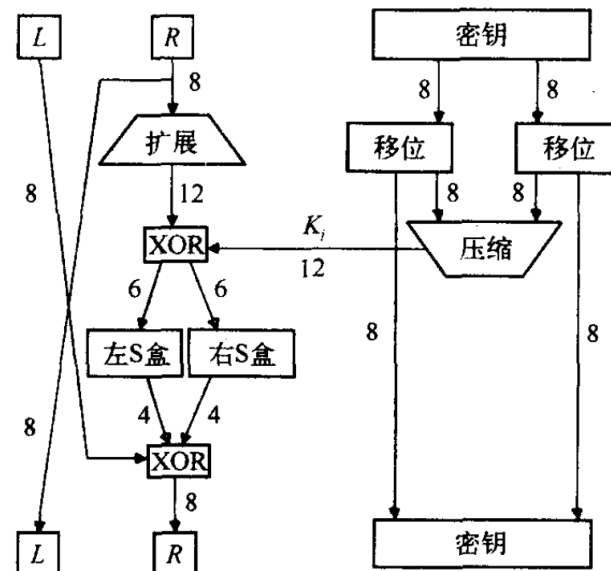
- TDES 是一种Feistel密码。
- 明文: (L_0, R_0)
- 对于 $i = 1$ 到 4

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

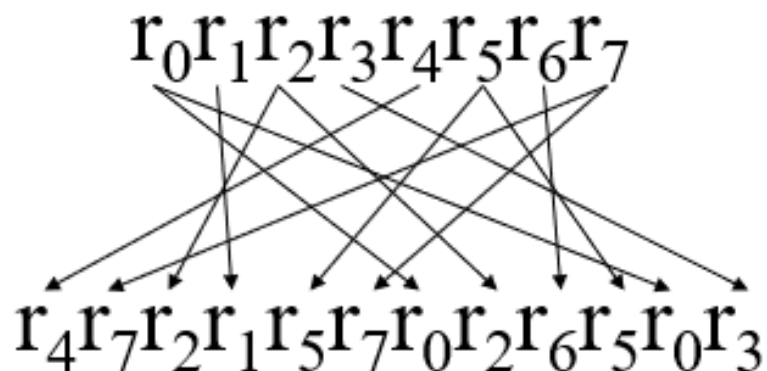
- 密文: (L_4, R_4)
- 轮函数: $F(R_{i-1}, K_i) = \text{Sboxes}(\text{expand}(R_{i-1}) \boxtimes K_i)$
 - 其中 $\text{Sboxes}(x_0x_1x_2 \dots x_{11}) = (\text{SboxLeft}(x_0x_1 \dots x_5), \text{SboxRight}(x_6x_7 \dots x_{11}))$

TDES的一轮



TDES 扩展置换

- 扩展排列：8 位到 12 位



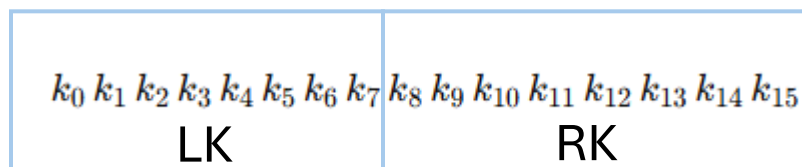
□ 我们可以这样写：

$$\text{expand}(r_0 r_1 r_2 r_3 r_4 r_5 r_6 r_7) = r_4 r_7 r_2 r_1 r_5 r_7 r_0 r_2 r_6 r_5 r_0 r_3$$

微型DES密钥

- 密钥在四轮中的使用值:

TDES密钥



LK循环左移2位

RK循环左移1位

$$K_i = k_0 k_2 k_3 k_4 k_5 k_7 k_9 k_{10} k_{11} k_{13} k_{14} k_{15}$$

$$K_1 = k_2 k_4 k_5 k_6 k_7 k_1 k_{10} k_{11} k_{12} k_{14} k_{15} k_8$$

$$K_2 = k_4 k_6 k_7 k_0 k_1 k_3 k_{11} k_{12} k_{13} k_{15} k_8 k_9$$

$$K_3 = k_6 k_0 k_1 k_2 k_3 k_5 k_{12} k_{13} k_{14} k_8 k_9 k_{10}$$

$$K_4 = k_0 k_2 k_3 k_4 k_5 k_7 k_{13} k_{14} k_{15} k_9 k_{10} k_{11}$$

TDES S盒

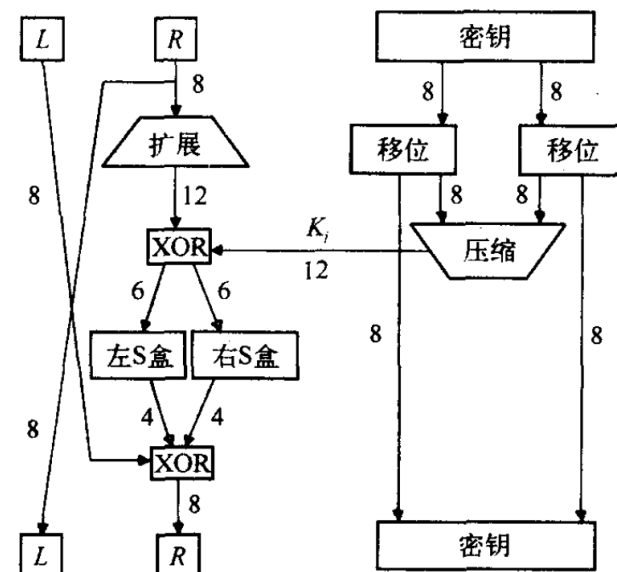
- 计算F轮函数: $F(R, K) = (Sbox(extend(R) \oplus K) = SboxLeft(x_0x_1x_2x_3x_4x_5), SboxRight(x_6x_7x_8x_9x_{10}x_{11}))$

TDES 左S盒

x_0x_5	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	6	9	A	3	4	D	7	8	E	1	2	B	5	C	F	0
1	9	E	B	A	4	5	0	7	8	6	3	2	C	D	1	F
2	8	1	C	2	D	3	E	F	0	9	5	A	4	B	6	7
3	9	0	2	5	A	D	6	E	1	8	B	C	3	4	7	F

TDES 右S盒

x_0x_5	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	C	5	0	A	E	7	2	8	D	4	3	9	6	F	1	B
1	1	C	9	6	3	E	B	2	F	8	4	5	D	A	0	7
2	F	A	E	6	D	8	2	4	1	7	9	0	3	5	B	C
3	0	A	3	C	8	2	1	E	9	7	F	6	B	5	D	4



TDES差分分析

- 这里TDES差分分析集中在S右盒上。
- 针对 $X_1 \oplus X_2 = 001000$ 的 X_1, X_2 , 计算右盒S盒的输出差分:
- $X_1 \oplus X_2 = 001000$ 对应输出
 $SboxRight(X_1) \oplus SboxRight(X_2) = 0010$ 的概率是 $3/4$ 。
- 对于任意相同输入有相同输出: $X_1 \oplus X_2 = 000000$ 对应输出
 $SboxRight(X_1) \oplus SboxRight(X_2) = 0000$.
- 我们使用这些概率结果构造出可行的差分分析。

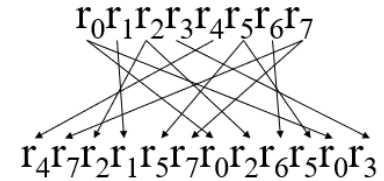
TDES差分分析

- 差分分析是选择明文攻击：
- 选择 P 和 P' ，以便

$$P \oplus P' = 0000\ 0000\ 0000\ 0010 = 0x0002$$

- 请注意， P 和 P' 相差恰好1位。
- 让我们仔细分析一下当这些明文使用 TDES 加密时会发生什么。

TDES差分分析



- 如果 $Y \oplus Y = 001000$, 则概率为 $3/4$

$$\text{SboxRight}(Y) \oplus \text{SboxRight}(Y) = 0010$$
- $Y \oplus Y = 001000 \Rightarrow (Y \oplus K) \oplus (Y \oplus K) = 001000$
- 如果 $Y \oplus Y = 000000$, 则对于任意 S 盒, 我们有 $\text{Sbox}(Y) \oplus \text{Sbox}(Y) = 0000$
- 差值(0000 0010)通过 TDES expand perm 扩展为 diff. (000000 001000)
- **结论:** 如果 $X \oplus X = 00000010$, 则 $F(X, K) \oplus F(X, K) = 00000010$, 概率为 $3/4$

TDES差分分析

- 上一张幻灯片
 - 假设 $R \oplus R = 0000\ 0010$
 - 假设 K 是未知密钥
 - 那么, 以 $3/4$ 的概率
$$F(R, K) \subseteq F(R, K) = 0000\ 0010$$
- 结论是什么? 有 $3/4$ 的概率.....
 - 下一轮的输入与本轮相同
- 所以我们可以连续进行多轮。

TDES差分分析

- TDES差分分析四轮迭代的对应结果
- 选择P和 $\tilde{P}$ ，其中 $P \oplus \tilde{P} = 0X0002$

$(L_0, R_0) = P$	$(\tilde{L}_0, \tilde{R}_0) = \tilde{P}$	$P \oplus \tilde{P} = 0x0002$	概率
$L_1 = R_0$	$\tilde{L}_1 = \tilde{R}_0$		
$R_1 = L_0 \oplus F(R_0, K_1)$	$\tilde{R}_1 = \tilde{L}_0 \oplus F(\tilde{R}_0, K_1)$	$(L_1, R_1) \oplus (\tilde{L}_1, \tilde{R}_1) = 0x0202$	3/4
$L_2 = R_1$	$\tilde{L}_2 = \tilde{R}_1$		
$R_2 = L_1 \oplus F(R_1, K_2)$	$\tilde{R}_2 = \tilde{L}_1 \oplus F(\tilde{R}_1, K_2)$	$(L_2, R_2) \oplus (\tilde{L}_2, \tilde{R}_2) = 0x0200$	$(3/4)^2$
$L_3 = R_2$	$\tilde{L}_3 = \tilde{R}_2$		
$R_3 = L_2 \oplus F(R_2, K_3)$	$\tilde{R}_3 = \tilde{L}_2 \oplus F(\tilde{R}_2, K_3)$	$(L_3, R_3) \oplus (\tilde{L}_3, \tilde{R}_3) = 0x0002$	$(3/4)^2$
$L_4 = R_3$	$\tilde{L}_4 = \tilde{R}_3$		
$R_4 = L_3 \oplus F(R_3, K_4)$	$\tilde{R}_4 = \tilde{L}_3 \oplus F(\tilde{R}_3, K_4)$	$(L_4, R_4) \oplus (\tilde{L}_4, \tilde{R}_4) = 0x0202$	$(3/4)^3$
$C = (L_4, R_4)$	$\tilde{C} = (\tilde{L}_4, \tilde{R}_4)$	$C \oplus \tilde{C} = 0x0202$	

TDES差分分析

- 选择明文P和 $\tilde{P}$ ，其中 $P \oplus \tilde{P} = 0X0002 = 0000\ 0000\ 0000\ 0010$
- 这样左侧S盒的输入差分为0，所以输出差分为0. 只考虑右侧S盒。
- 右侧： $R \oplus \tilde{R} = 0000\ 0010$ ，那么对任意子密钥K，有：

$$F(R, K) \oplus F(\tilde{R}, K) = 0000\ 0010$$
- 的概率为3/4.
- 其中： $R_0 \oplus \tilde{R}_0 = 0000\ 0010$ 和 $L_0 \oplus \tilde{L}_0 = 0000\ 0000$
- 可得： $R_1 \oplus \tilde{R}_1 = 0000\ 0010$ 的概率为3/4。
- 对应： $R_2 \oplus \tilde{R}_2 = (L_1 \oplus F(R_1, K_2)) \oplus (\tilde{L}_1 \oplus F(\tilde{R}_1, K_2))$ 的概率为 $(3/4)^2$

$$= (L_1 \oplus \tilde{L}_1) \oplus (F(R_1, K_2) \oplus F(\tilde{R}_1, K_2))$$

$$= (R_0 \oplus \tilde{R}_0) \oplus (F(R_1, K_2) \oplus F(\tilde{R}_1, K_2))$$

$$= 0000\ 0010 \oplus 0000\ 0010$$

$$= 0000\ 0000$$

TDES差分分析

轮数/状态	左侧异或值 ΔL_i	右侧异或值 ΔR_i	左右合并异或值
初始明文对 $P, \tilde{P}$	0000	0010	0x0002
第1轮后 (L_1, R_1)	0010	0010	0x0202
第2轮后 (L_2, R_2)	0010	0000	0x0200
第3轮后 (L_3, R_3)	0000	0010	0x0002
第4轮后 $(L_4, R_4) = C$	0010	0010	0x0202

TDES差分分析

- 选择P和 $\tilde{P}$ ，其中 $P \oplus \tilde{P} = 0x0002$

- 如果 $C \oplus \tilde{C} = 0x0202$ 时

$$R_4 = L_3 \oplus F(R_3, K_4) \text{ 和 } \tilde{R}_4 = \tilde{L}_3 \oplus F(\tilde{R}_3, K_4)$$

- 并且有： $L_4 = R_3$ 和 $\tilde{L}_4 = \tilde{R}_3$

- 所以： $R_4 = L_3 \oplus F(L_4, K_4)$ 和 $\tilde{R}_4 = \tilde{L}_3 \oplus F(\tilde{L}_4, K_4)$

- 现在有 $C \oplus \tilde{C} = 0x0202$

- 所以根据前面的表格有： $L_3 \oplus \tilde{L}_3 = 0000 \ 0000$

- 所以上式可以变成 $R_4 \oplus F(L_4, K_4) = \tilde{R}_4 \oplus F(\tilde{L}_4, K_4)$

$$R_4 \oplus \tilde{R}_4 = F(L_4, K_4) \oplus F(\tilde{L}_4, K_4)$$

TDES差分分析

- 我们根据公式可以恢复出子密钥的某些位: $R_4 \oplus \tilde{R}_4 = F(L_4, K_4) \oplus F(\tilde{L}_4, K_4)$

$$C \oplus \tilde{C} = (L_4, R_4) \oplus (\tilde{L}_4, \tilde{R}_4) = 0x0202$$

$$L_4 \oplus \tilde{L}_4 = 0000\ 0010 \quad R_4 \oplus \tilde{R}_4 = 0000\ 0010$$

$$L_4 = \ell_0\ell_1\ell_2\ell_3\ell_4\ell_5\ell_6\ell_7 \text{ 和 } \tilde{L}_4 = \tilde{\ell}_0\tilde{\ell}_1\tilde{\ell}_2\tilde{\ell}_3\tilde{\ell}_4\tilde{\ell}_5\tilde{\ell}_6\tilde{\ell}_7$$

$$\begin{aligned} 0000\ 0010 = & \left(\text{SboxLeft}(\ell_4\ell_7\ell_2\ell_1\ell_5\ell_7 \oplus k_0k_2k_3k_4k_5k_7), \right. \\ & \left. \text{SboxRight}(\ell_0\ell_2\ell_6\ell_5\ell_0\ell_3 \oplus k_{13}k_{14}k_{15}k_9k_{10}k_{11}) \right) \\ & \oplus \left(\text{SboxLeft}(\tilde{\ell}_4\tilde{\ell}_7\tilde{\ell}_2\tilde{\ell}_1\tilde{\ell}_5\tilde{\ell}_7 \oplus k_0k_2k_3k_4k_5k_7), \right. \\ & \left. \text{SboxRight}(\tilde{\ell}_0\tilde{\ell}_2\tilde{\ell}_6\tilde{\ell}_5\tilde{\ell}_0\tilde{\ell}_3 \oplus k_{13}k_{14}k_{15}k_9k_{10}k_{11}) \right) \end{aligned}$$

TDES差分分析

- 从而得出左侧位:

$$\begin{aligned} 0000 &= \text{SboxLeft}(l_4 l_7 l_2 l_1 l_5 l_7 \oplus k_0 k_2 k_3 k_4 k_5 k_7) \\ &\quad \oplus \text{SboxLeft}(\tilde{l}_4 \tilde{l}_7 \tilde{l}_2 \tilde{l}_1 \tilde{l}_5 \tilde{l}_7 \oplus k_0 k_2 k_3 k_4 k_5 k_7), \end{aligned}$$

- 对于任意选择的密钥位 $k_i (i \neq 6)$ 成立 $l_i = \tilde{l}_i$ 。
- 右侧位:
$$\begin{aligned} 0010 &= \text{SboxRight}(l_0 l_2 l_6 l_5 l_0 l_3 \oplus k_{13} k_{14} k_{15} k_9 k_{10} k_{11}) \\ &\quad \oplus \text{SboxRight}(\tilde{l}_0 \tilde{l}_2 \tilde{l}_6 \tilde{l}_5 \tilde{l}_0 \tilde{l}_3 \oplus k_{13} k_{14} k_{15} k_9 k_{10} k_{11}) \end{aligned}$$
- 必须正确选择子密钥位, 才能能使上式成立。

TDES差分分析

- 使用回复子密钥算法对满足 P 和 $\tilde{P} = 0x0002$ 进行测试。
- 通过此实验可以初步确定：
- 第四轮子密钥 K_4

表 6.4 恢复子密钥位算法

$k_{13}k_{14}k_{15}k_9k_{10}k_{11} \in$
 $\{000001, 001001, 110000, 111000\}$

实际密钥为

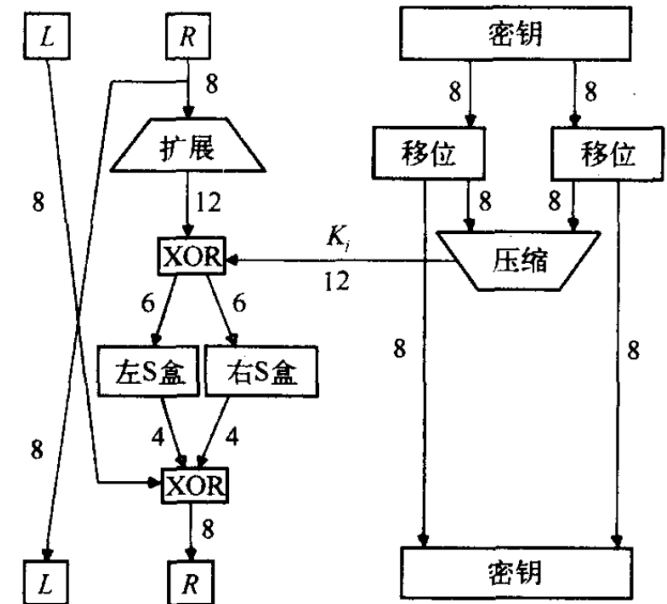
$K=1010\ 1001\ 1000\ 0111$

```

count[i] = 0, for i = 0, 1, ..., 63
for i = 1 迭代次数
    选择 P 和  $\tilde{P}$  满足  $P \oplus \tilde{P} = 0x0002$ 
    获得相应的  $C = c_0c_1 \dots c_{15}$  and  $\tilde{C} = \tilde{c}_0\tilde{c}_1 \dots \tilde{c}_{15}$ 
    if  $C \oplus \tilde{C} = 0x0202$  then
         $\ell_i = c_i$  and  $\tilde{\ell}_i = \tilde{c}_i$  for  $i = 0, 1, \dots, 7$ 
        for K = 0 to 63
            if  $0010 == (SboxRight(\ell_0\ell_2\ell_6\ell_5\ell_0\ell_3 \oplus K)$ 
                 $\oplus SboxRight(\tilde{\ell}_0\tilde{\ell}_2\tilde{\ell}_6\tilde{\ell}_5\tilde{\ell}_0\tilde{\ell}_3 \oplus K))$  then
                    increment count[K]
            end if
        next K
    end if
next i
    
```

TDES线性分析

- TDES的差分分析主要集中在**右S盒**
- TDES的线性分析主要集中在**左S盒**
- 线性分析属于**已知明文攻击**。



- 符号表示: $y_0 y_1 y_2 y_3 = \text{SboxLeft}(x_0 x_1 x_2 x_3 x_4 x_5)$
- 对于这个S盒, $y_1 = x_2$ 和 $y_2 = x_3$ 的概率均为 $3/4$

TDES线性分析

- 回想一下，扩展永久式是

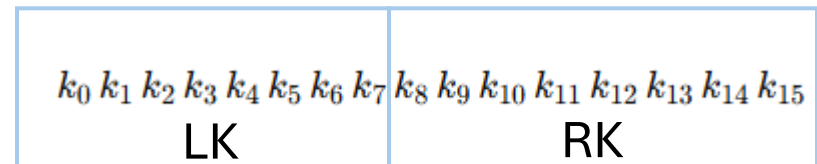
$$\text{expand}(r_0 r_1 r_2 r_3 r_4 r_5 r_6 r_7) = r_4 r_7 \mathbf{r_2} \mathbf{r_1} r_5 r_7 r_0 r_2 r_6 r_5 r_0 r_3$$

- $y_0 y_1 y_2 y_3 = \text{SboxLeft}(x_0 x_1 x_2 x_3 x_4 x_5)$ ，其中 $y_1 = x_2$ 且 $y_2 = x_3$ 的概率均为3/4.
- 此外， $\text{expand}(R_{i-1}) \oplus K_i$ 在第i轮
- 那么 $y_1 = r_2 \oplus k_5$ 和 $y_2 = r_1 \oplus k_n$ 都具有概率 3/4.
- 新的右半部分是 $y_0 y_1 y_2 y_3 \dots$ 加上原来的左半部分
- **结论：**新的右半部分： $r_1 \leftarrow r_2 \oplus k_m \oplus l_1$ 和 $r_2 \leftarrow r_1 \oplus k_n \oplus l_2$ 两者概率均为3/4

TDES线性分析

- TDES密钥:
- 密钥在四轮中的使用值:

TDES密钥



LK循环左移2位



RK循环左移1位



$$K_i = k_0 k_2 k_3 k_4 k_5 k_7 k_9 k_{10} k_{11} k_{13} k_{14} k_{15}$$



$$K_1 = k_2 k_4 k_5 k_6 k_7 k_1 k_{10} k_{11} k_{12} k_{14} k_{15} k_8$$

$$K_2 = k_4 k_6 k_7 k_0 k_1 k_3 k_{11} k_{12} k_{13} k_{15} k_8 k_9$$

$$K_3 = k_6 k_0 k_1 k_2 k_3 k_5 k_{12} k_{13} k_{14} k_8 k_9 k_{10}$$

$$K_4 = k_0 k_2 k_3 k_4 k_5 k_7 k_{13} k_{14} k_{15} k_9 k_{10} k_{11}$$

TDES线性分析

• TDES线性分析表

$(L_0, R_0) = (p_0 \dots p_7, p_8 \dots p_{15})$	第 1 位和第 2 位(索引从 0 开始)	概率
$L_1 = R_0$	p_9, p_{10}	1
$R_1 = L_0 \oplus F(R_0, K_1)$	$p_1 \oplus p_{10} \oplus k_5, p_2 \oplus p_9 \oplus k_6$	$3/4$
$L_2 = R_1$	$p_1 \oplus p_{10} \oplus k_5, p_2 \oplus p_9 \oplus k_6$	$3/4$
$R_2 = L_1 \oplus F(R_1, K_2)$	$p_2 \oplus k_6 \oplus k_7, p_1 \oplus k_5 \oplus k_0$	$(3/4)^2$
$L_3 = R_2$	$p_2 \oplus k_6 \oplus k_7, p_1 \oplus k_5 \oplus k_0$	$(3/4)^2$
$R_3 = L_2 \oplus F(R_2, K_3)$	$p_{10} \oplus k_0 \oplus k_1, p_9 \oplus k_7 \oplus k_2$	$(3/4)^3$
$L_4 = R_3$	$p_{10} \oplus k_0 \oplus k_1, p_9 \oplus k_7 \oplus k_2$	$(3/4)^3$
$R_4 = L_3 \oplus F(R_3, K_4)$		
$C = (L_4, R_4)$	$c_1 = p_{10} \oplus k_0 \oplus k_1, c_2 = p_9 \oplus k_7 \oplus k_2$	$(3/4)^3$

TDES线性分析

- 实验结果
- 使用100个已知的明文，得到密文。
 - 令 $P = p_0 p_1 p_2 \dots p_{15}$ ，令 $C = c_0 c_1 c_2 \dots c_{15}$
- 统计结果
 - $c_1 \oplus p_{10} = 0$ 出现38次
 - $c_1 \oplus p_{10} = 1$ 出现62次
 - $c_2 \oplus p_9 = 0$ 出现62次
 - $c_2 \oplus p_9 = 1$ 出现38次
- 结论
 - 因为 $k_0 \oplus k_1 = c_1 \oplus p_{10}$ 我们有 $k_0 \oplus k_1 = 1$
 - 因为 $k_7 \oplus k_2 = c_2 \oplus p_9$ 我们有 $k_7 \oplus k_2 = 0$
- 实际密钥为 $K = 1010\ 0011\ 0101\ 0110$

为了构建更好的分组密码.....

- 密码学家如何才能使线性攻击和差分分析更加困难?
- **关键点：** 轮数，混淆的程度和扩散的数量。
 1. **更多回合** —— 每轮成功概率递减
 2. **更好的混淆** (S盒) —— 降低每一轮的成功概率
 3. **更好的扩散** (排列组合) —— 更难进行多轮连锁反应
- 混合程度和非线性程度有限，意味着需要更多轮次：
TEA
- 强混合和非线性， 然后进行更少轮迭代： AES

RSA의侧信道攻击 (旁门攻击)

- 有时无需直接攻击加密算法即可恢复密钥。
- 许多处理过程或产生信息泄露：
 - 计算的执行方式
 - 使用的介质
 - 能量的消耗
 - 电磁泄漏
- 诱导故障也能泄露信息
- 旁门信息可能泄露加密密钥
- Paul Kocher是该领域的带头人。

侧信道类型

- 电磁辐射安全 (EMSEC)
 - 计算机屏幕产生的电磁场 (EMF) 可以让屏幕图像在一定距离外重建。
 - 智能卡曾遭受电磁场辐射攻击
- 差分功率分析 (DPA)
 - 智能卡的功耗取决于计算能力。
- 差分故障分析 (DFA)
 - GSM系统中存储在智能卡上的密钥可以通过闪光灯读取，从而诱发故障。
- 时序分析
 - 不同的计算需要不同的时间
 - 通过网络恢复的 RSA 密钥

场景

- Alice 的公钥: (N, e)
- Alice 的私钥: d
- Trudy 想找到 d 。
- Trudy 可以向 Alice 发送任意消息 M ，Alice 将回复 $M^d \bmod N$
 - 也就是说，Alice 签署 M 并将结果发送给 Trudy。
- Trudy 可以精确地计算 Alice 计算 $M^d \bmod N$ 所需的时间。

RSA 的时间攻击

- 考虑 $M^d \bmod N$
- 我们想找到私钥 d ,
其中 $d = d_0 d_1 \dots d_n$
- 用于 $M^d \bmod N$ 的重复平方
- 假设为了效率:

$\text{mod}(x, N)$

if $x \geq N$

$x = x \% N$

end if

return x

重复平方算法

$x = M$

for $j = 1$ to n

$x = \text{mod}(x^2, N)$

if $d_j == 1$ then

$x = \text{mod}(x * M, N)$

end if

next j

return x

RSA 的时间攻击

- 如果 $d_j = 0$ 则
 - $x = \text{mod}(x^2, N)$
- 如果 $d_j = 1$ 则
 - $x = \text{mod}(x^2, N)$
 - $x = \text{mod}(x * M, N)$
- 计算时间因情况而异
- 攻击者能否利用这一点?

重复平方

```
x = M
for j = 1 to n
    x = mod(x2, N)
    if dj == 1 then
        x = mod(x * M, N)
    end if
next j
return x
mod(x, N)
if x >= N
    x = x % N
end if
return x
```

RSA 的时间攻击

- 选择 M , 其中 $M^3 < N$
- 选择 M 满足 $M^2 < N < M^3$
- 令 $x = M$ 且 $x = M$
- 考虑 $j = 1$
 - $x = \text{mod}(x^2, N)$ 没有 “%”
 - $x = \text{mod}(x * M, N)$ 没有 “%”
 - $x = \text{mod}(x^2, N)$ 没有 “%”
 - $x = \text{mod}(x * M, N)$ 仅在 $d_1 = 1$ 时执行 “%”
- 如果 $d_1 = 1$, 则对于 M 而言, $j = 1$ 步所需时间比对于 M 而言更长。
- 但不止一轮.....

重复平方

$x = M$

for $j = 1$ to n

$x = \text{mod}(x^2, N)$

if $d_j == 1$ then

$x = \text{mod}(x * M, N)$

end if

next j

return x

def **mod**(x, N)

if $x \geq N$

$x = x \% N$

end if

return x

RSA 的时间攻击

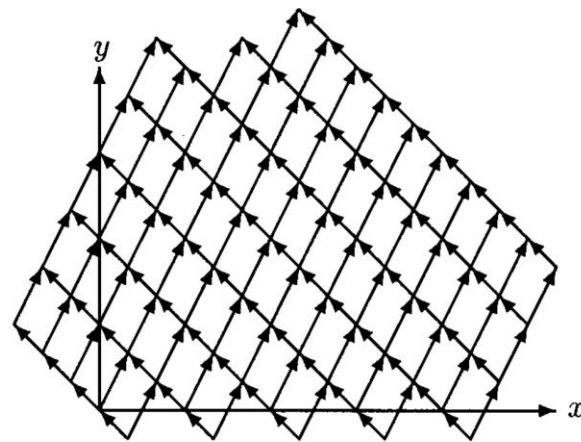
- 选择明文攻击的一个例子
- 选择 $M_0, M_1, \dots, M_{m-1}$
 - $M_i^3 < N$ 对于 $i = 0, 1, \dots, m-1$
- 令 t_i 为计算 $M_i^d \bmod N$ 所需的时间
 - $t = (t_0 + t_1 + \dots + t_{m-1}) / m$
- 选择 $M_0, M_1, \dots, M_{m-1}$
 - 对于 $i = 0, 1, \dots, m-1$, $M_i^2 < N < M_i^3$
- 令 t_i 为计算 $M_i^d \bmod N$ 所需的时间
 - $t = (t_0 + t_1 + \dots + t_{m-1}) / m$
- 如果 $t > t$ 则 $d_1 = 1$, 否则 $d_1 = 0$
- 一旦知道了 d_1 , 就求出 d_2 , 然后求出 d_3 , 以此类推.....

侧信道攻击

- 侧信道攻击是重要的一类解密类型
- 密码在理论上是安全的，在实践中不一定安全
- 孤立的分析密码不够
- 需要结合特殊的软硬件环境
- 教训： **攻击者不按规则行事！**

背包密码的格约简攻击

- **晶格**中找到一个“短”矢量，可以解决许多问题。
- 设 $b_1, b_2, \dots, b_n$ 为 $\mathbb{R}^m$ 中的向量
- 所有 $\alpha_1 b_1 + \alpha_2 b_2 + \dots + \alpha_n b_n$,
其中每个 α_i 为整数，是一个离散点集。

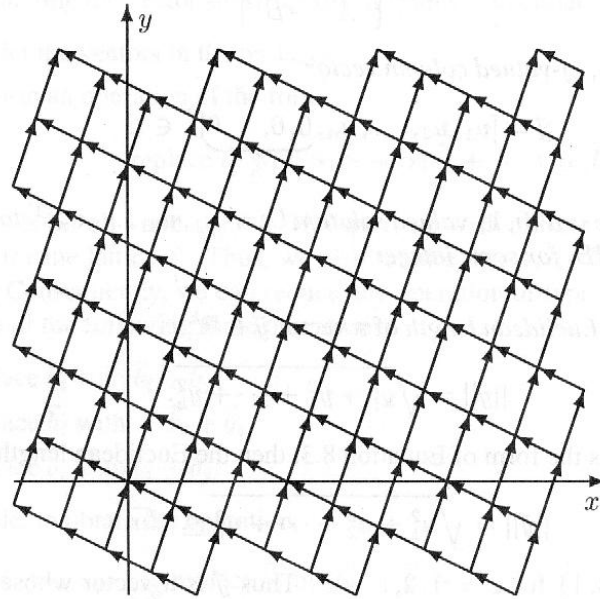


什么是晶格?

- 假设 $b_1 = [1, 3]^T$ 且 $b_2 = [-2, 1]^T$
- 那么平面上的任意一点都可以表示为 $\alpha_1 b_1 + \alpha_2 b_2$, 其中 $\alpha_1, \alpha_2 \in \mathbb{R}$.
 - 由于 b_1 和 b_2 线性无关。
- 我们称 (b_1, b_2) 范围在平面 $\mathbb{R}^2$ 上
- 如果 α_1 和 α_2 被限制为整数，则所得的跨度是一个格
- 那么，格就是一个离散点集。

晶格示例

- 假设 $b_1 = [1, 3]^T$ 且 $b_2 = [-2, 1]^T$
- (b_1, b_2) 张成的晶格如图右侧所示。



恰好覆盖

- **完全覆盖** —— 给定一个集合 S 和 S 的若干子集，找到这些子集的集合，使得 S 中的每个元素都恰好属于其中一个子集。
- 精确覆盖问题可以通过在格中找到一个“短”向量来解决。

恰好覆盖示例

- 集合 $S = \{0, 1, 2, 3, 4, 5, 6\}$
- 假设 $m = 7$ 个元素, $n = 13$ 个子集
子集: 0 1 2 3 4 5 6 7 8 9 10 11 12
元素: 013 015 024 025 036 124 126 135 146 1 256 345 346
- 找到一组这样的子集, 使得集合 S 中的每个元素都恰好属于其中一个子集。
- 可以尝试所有 2^{13} 种可能性
- 如果问题太大, 可以尝试 **启发式搜索**。
- 许多不同的启发式搜索技术

恰好覆盖解决方案

- 矩阵形式的精确覆盖
 - 集合 $S = \{0, 1, 2, 3, 4, 5, 6\}$
 - 假设 $m = 7$ 个元素, $n = 13$ 个子集

子集: 0 1 2 3 4 5 6 7 8 9 10 11 12

元素: 013 015 024 025 036 124 126 135 146 1 256 345 346

$$\begin{array}{c} \text{元素} \end{array}
 \begin{array}{c} \text{子集} \end{array}
 \begin{array}{c} m \times n \end{array}
 \begin{array}{c} n \times 1 \end{array}
 =
 \begin{array}{c} m \times 1 \end{array}$$

The matrix equation is represented as:

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}
 \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \\ u_8 \\ u_9 \\ u_{10} \\ u_{11} \\ u_{12} \end{bmatrix}
 =
 \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

解: $AU = B$

这里 $u_i \in \{0, 1\}$

解决方案:

$$U = [0001000001001]^T$$

恰好覆盖解决方案

- 我们可以将 $AU = B$ 重述为 $MV = W$, 其中

$$\begin{bmatrix} I_{n \times n} & 0_{n \times 1} \\ A_{m \times n} & -B_{m \times 1} \end{bmatrix} \begin{bmatrix} U_{n \times 1} \\ 1_{1 \times 1} \end{bmatrix} = \begin{bmatrix} U_{n \times 1} \\ 0_{m \times 1} \end{bmatrix} \iff AU = B$$

矩阵M 向量V 向量W

- 期望的解是U
 - M的列向量线性无关
- 设 $c_0, c_1, c_2, \dots, c_n$ 为矩阵M的n+1个列
- 设 $v_0, v_1, v_2, \dots, v_n$ 为V
- 那么 $W = v_0 c_0 + v_1 c_1 + \dots + v_n c_n$

恰好覆盖解决方案

- 令 L 为由 $c_0, c_1, c_2, \dots, c_n$ 张成的格。 (c_i 是 M 的列)
- 回想一下, $MV = W$
 - 其中 $W = [U, 0]^T$, 我们想要找到 U
 - 但是, 如果我们找到了 W , 我们也解出了这个问题!
- 注意, 由于所有 v_i 均为整数, 且 $W = v_0 c_0 + v_1 c_1 + \dots + v_n c_n$

恰好覆盖解决方案

- $W = [u_0, u_1, \dots, u_{n-1}, 0, 0, \dots, 0] \in L$, 每个 $u_i \in \{0, 1\}$
- $Y \in \mathbb{R}^n$ 的长度是

$$\|Y\| = \sqrt{y_0^2 + y_1^2 + \dots + y_{N-1}^2}$$

- W 的长度为

$$\|W\| = \sqrt{u_0^2 + u_1^2 + \dots + u_{n-1}^2} \leq \sqrt{n}$$

- 因此, W 是 L 中一个非常短的向量, 其中
 - W 的前 n 个元素均为 0 或 1
 - W 的最后 m 个元素均为 0。
- 我们能否利用这些事实求出 U ?

LLL算法

- LLL算法可以有效地找出格中的短向量。

```
// 找出格中的短向量
// 通过  $M$  的列  $= (b_0, b_1, \dots, b_n)$ 
loop forever
   $(X, Y) = \text{GS}(M)$ 
  for  $j = 1$  to  $n$ 
    for  $i = j - 1$  to  $0$ 
      if  $|y_{ij}| > 1/2$  then
         $b_j = b_j - \lfloor y_{ij} + 1/2 \rfloor b_i$ 
      end if
    next  $i$ 
  next  $j$ 
   $(X, Y) = \text{GS}(M)$ 
  for  $j = 0$  to  $n - 1$ 
    if  $\|x_{j+1} + y_{j,j+1}x_j\|^2 < \frac{3}{4}\|x_j\|^2$ 
      swap( $b_j, b_{j+1}$ )
      goto abc
    end if
  next  $j$ 
  return( $M$ )
abc:
  continue
end loop
```

背包密码示例

- 格约简与背包密码系统有什么关系?
- 假设我们有
 - 超级递增背包

$$S = [2, 3, 7, 14, 30, 57, 120, 251]$$

- 假设 $m = 41$, $n = 491 \Rightarrow m^{-1} = 12 \pmod{n}$
- 公共背包: $t_i = 41 \cdot s_i \pmod{491}$

$$T = [82, 123, 287, 83, 248, 373, 10, 471]$$

- **公钥:** T **私钥:** (S, m^{-1}, n)

背包密码示例

• 公钥: T 私钥: (S, m^{-1}, n)

$S = [2, 3, 7, 14, 30, 57, 120, 251]$

$T = [82, 123, 287, 83, 248, 373, 10, 471]$

$n = 491, m^{-1} = 12$

• 例如: 10010110加密为

$82 + 83 + 373 + 10 = 548$

• 然后接收器进行计算。

$548 \times 12 = 193 \pmod{491}$

并利用 S 求解10010110

背包 LLL算法 攻击

- 攻击者知道公钥

$$T = [82, 123, 287, 83, 248, 373, 10, 471]$$

- 攻击者知道密文： 548
- 攻击者想找到 $u_i \in \{0, 1\}$ 使得

$$82u_0 + 123u_1 + 287u_2 + 83u_3 + 248u_4 + 373u_5 + 10u_6 + 471u_7 = 548$$

- 这可以写成矩阵方程（点积）： $T \cdot U = 548$

背包 LLL 算法攻击

- 攻击者知道: $T = [82, 123, 287, 83, 248, 373, 10, 471]$
- 想要求解: $T \cdot U = 548$, 其中每个 $u_i \in \{0, 1\}$

$$M = \begin{bmatrix} I_{8 \times 8} & 0_{8 \times 1} \\ T_{1 \times 8} & -C_{1 \times 1} \end{bmatrix} = \left[\begin{array}{cccccccc|c} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ \hline 82 & 123 & 287 & 83 & 248 & 373 & 10 & 471 & -548 \end{array} \right]$$

□ M的列向量张成的晶格中的短向量。

LLL 算法结果

- 攻击者在M晶格里找到最短向量
- 矩阵M' 是应用 LLL 对M

$$M' = \left[\begin{array}{cccccccc|c} -1 & -1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & -1 & 1 & 0 & 1 & -1 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 & 0 & -1 & 1 & 2 \\ 1 & -1 & -1 & 1 & 0 & -1 & 0 & -1 & 0 \\ 0 & 0 & 1 & 0 & -2 & -1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & -1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & -1 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & -1 \\ \hline 1 & -1 & 1 & 0 & 0 & 1 & -1 & 2 & 0 \end{array} \right]$$

- 标有“*”的列格式正确
- 可能的解: $U = [1, 0, 0, 1, 0, 1, 1, 0]^T$
- 很容易验证这确实是明文。

结论

- 背包密码的格约简攻击非常简单高效。
- 密码系统只有在无人发现攻击手段的情况下才是安全的。
- 教训： **数学的进步可以破解密码系统！**

Hellman的时间-存储权衡攻击

- 时间-存储权衡攻击: Time-Memory Trade-Off (TMTO)
- TMTO: 通过预计算和有限存储来降低在线搜索时间的攻击方法。

方法	在线时间	存储	特点
暴力穷举	大	小	不需预计算
全查表	很小	很大	需要存全部映射
Hellman TM TO	中等	中等	用链折中时间和存储

Popcnt (人口计数)

- 人口计数函数popcnt:
- $\text{Popcnt}(x) = x$: 计算X的二进制表中1的个数
- $\text{Popcnt}(13) = \text{popcnt}(1101) = 3$
- 如何高效地计算 $\text{popcnt}(x)$?

Popcnt (人口计数)

- 按位与 (AND) 算法:

如果x是32位整数,
算法需要执行**32步**。

```
t = 0
for i = 0 to 31
    t = t + (x >> i) & 1
next i
```

- 预计算, 使用存储器保存结果 (TMTO):

对每个字节值预计算popcnt (y),
结果存数组p(y)里。

$$t = p[x \& 0xff] + p[(x \gg 8) \& 0xff] + p[(x \gg 16) \& 0xff] + p[(x \gg 24) \& 0xff]$$

如果x是32位整数,

TMTO算法需要**4步**。但是, 它需要256字节存储空间, 存储每个字节的计算结果。

TMTO基础知识

- 预计算
 - 一次性工作
 - 结果存储在表中
- 预计算结果用于加快后续每次计算的速度。
- 平衡“存储”与“时间”
- 一般来说，预计算量越大，需要的初始工作量和“内存”就越多，但后续每次计算所需的“时间”就越少。

分组密码表示法

- 考虑一种分组密码（对称密钥密码）

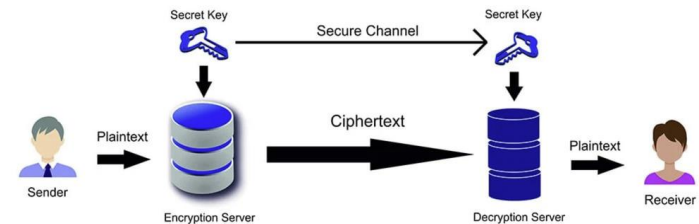
$$C = E(P, K)$$

这里：

P是大小为n

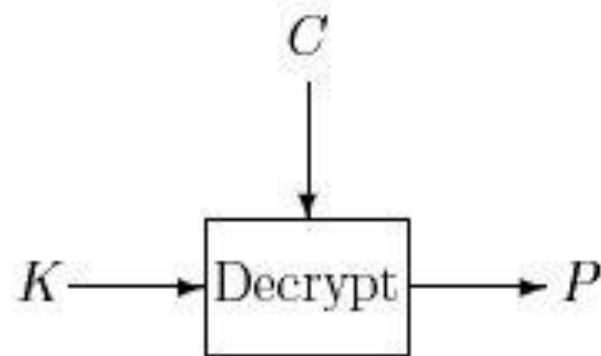
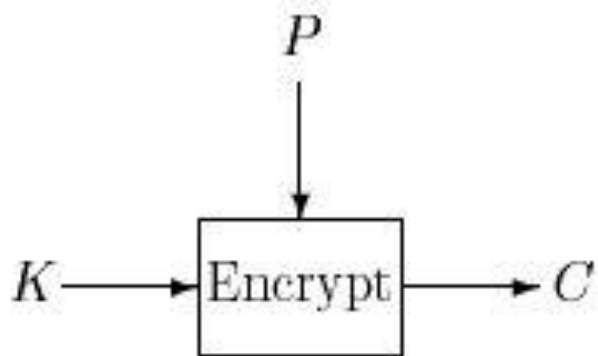
C是大小为n的密文块。

K是大小为k



Symmetric Cryptography

分组密码作为黑盒



- ❑ 对于 TMTO, 将分组密码视为黑盒
- ❑ 加密算法的细节并不重要

TMTO攻击

- **选择明文攻击**: 选择 P 并获得 C , 其中 $C = E(P, K)$
- 想找到关键的 K
- 两种“显而易见”的方法
 1. 穷举式密钥搜索
“内存”为0, 但每次攻击的“时间”为 2^k 。
 2. 预计算所有可能的 K 值对应的 $C = E(P, K)$
给定 C , 就可以直接在表中查找键 K 。
每次攻击的“内存”为 2^k , 但“时间”为0。
- TMTO 位于1和2 之间。

加密链

- 假设数据块长度和密钥长度相等: $n = k$
- 然后是一系列**加密**。

起点: SP

$$K_0 = SP$$

$$K_1 = E(P, SP)$$

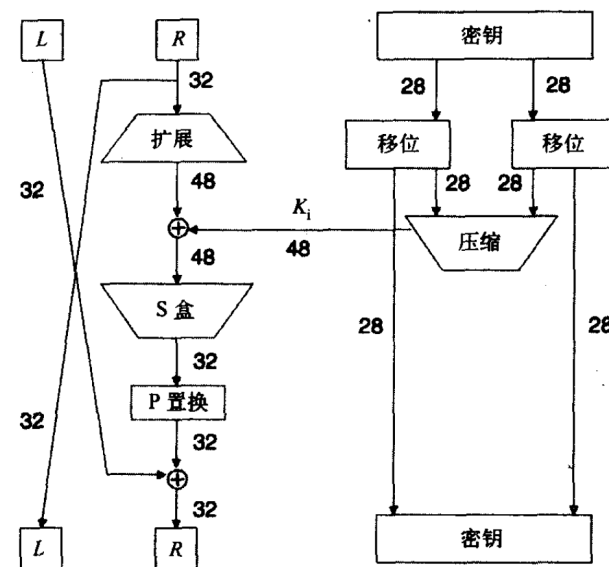
$$K_2 = E(P, K_1)$$

$$K_3 = E(P, K_2)$$

⋮

$$K_{t-1} = EP = E(P, K_{t-2})$$

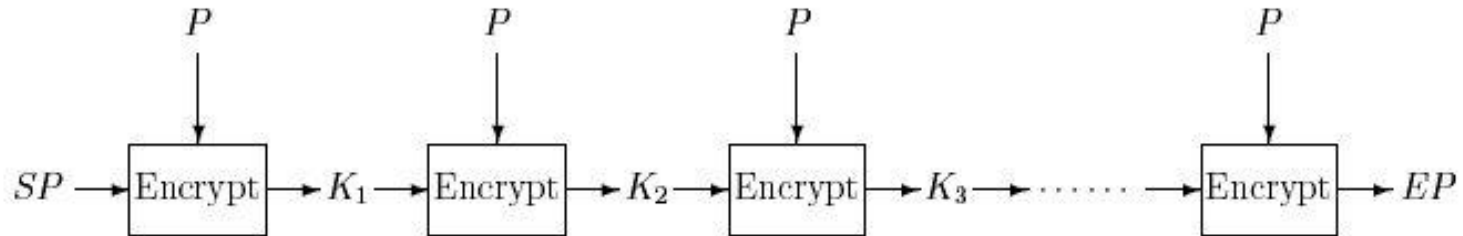
终点 EP



分组密码DES

加密链

- 密文用作下一次迭代的**密钥**
- 每次迭代都使用



预计算

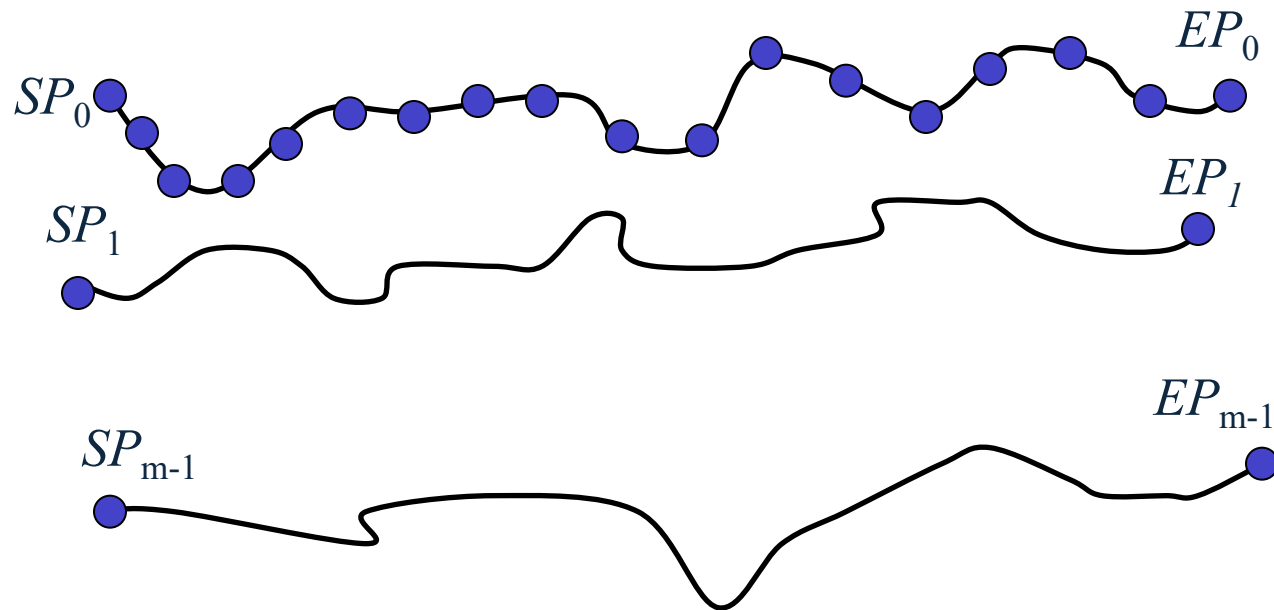
- 预先计算 m 条加密链，每条链的长度为 t_0+1
- 仅保存起点和终点

(SP_0, EP_0)

(SP_1, EP_1)

:

(SP_{m-1}, EP_{m-1})



TMTO攻击

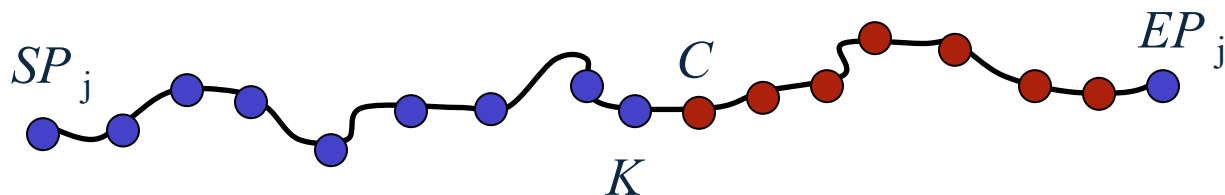
- **内存:** 预先计算加密链并保存 (SP_i, EP_i) $i = 0, 1, \dots, m-1$
 - 这是一次性工作
- 然后攻击一个特定的未知密钥 K
- 对于选定的相同的 P 用于查找加密链, 我们知道 C , 其中 $C = E(P, K)$, K 是未知密钥。
- **时间:** 计算链 (最多 t 步)
$$X_0 = C, X_1 = E(P, X_0), X_2 = E(P, X_1), \dots$$

TMTO攻击

- 考虑计算出的链

$$X_0 = C, X_1 = E(P, X_0), X_2 = E(P, X_1), \dots$$

- 假设对于某个 i , 我们发现 $X_i = EP_j$



- 自 $C = E(P, K)$ 链中键 K 在 C 之前!

TMTO攻击

- 综上所述，我们计算链

$$X_0 = C, X_1 = E(P, X_0), X_2 = E(P, X_1), \dots$$

- 如果对于某些 i ，我们发现 $X_i = EP_j$

- 然后就重建计算链 SP_j

$$Y_0 = SP_j, Y_1 = E(P, Y_0), Y_2 = E(P, Y_1), \dots$$

- 求 $C = Y_{t-i} = E(P, Y_{t-i-1})$ （总是吗？）

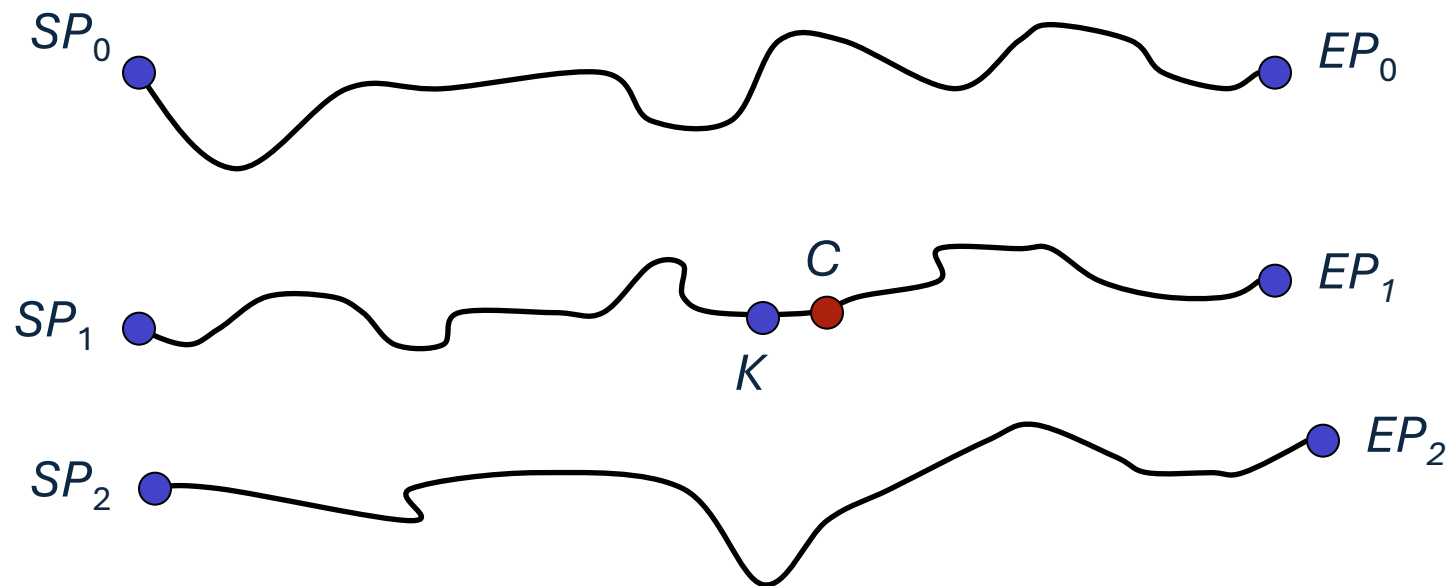
- 那么 $K = Y_{t-i-1}$ （总是吗？）

加密链理想情况

- 假设分组密码的 $k = 56$
 - 也就是说，密钥长度为56位。
- 假设我们找到 $m = 2^{28}$ 条链，每条链的长度为 $t = 2^{28}$ ，且链之间没有重叠。
- **记忆：** 2^{28} 对 (SP_j, EP_i)
- **时间：** 约 2^{28} （每次攻击）
 - C点出发，大约经过 2^{27} 步找到某个 EP_j 。
 - 用大约 2^{27} 步找到 K
- 攻击永不失败

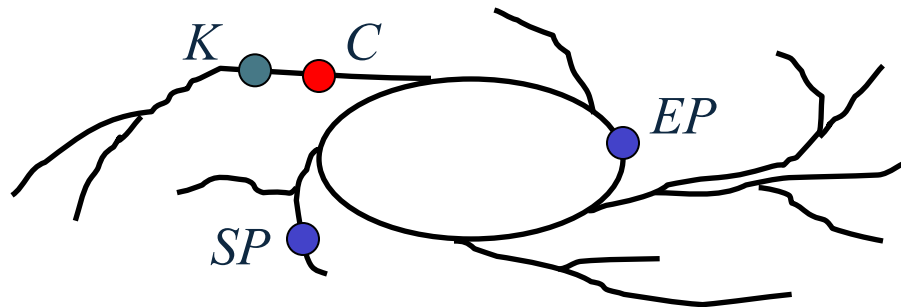
加密链理想情况

- 没有链重叠
- 任何密文 C 位于某个链中



现实世界的加密链

- 链条可没那么听话!
- 链条可以**循环**和**合并**



- ❑ C 链连接到 EP
- ❑ SP 到 EP 的链不包含 K

现实世界中的 TMT0 问题

- 合并、循环、误报等。
- 预计算工作量很大。
 - 必须多次攻击才能产生收益。
- 成功并非必然。
 - 概率取决于初始工作量
- 如果数据块大小不等于密钥长度怎么办？
 - 这很容易处理。
- 成功的概率是多少？
 - 这并非那么容易计算的。

现实世界的加密链

- 降低错误警报的数量：减少链的循环和合并
- 使用一个“随机函数”来计算链

$$K_0 = SP$$

$$K_1 = F(E(P, SP))$$

$$K_2 = F(E(P, K_1))$$

$$K_3 = F(E(P, K_2))$$

⋮

$$K_{t-1} = EP = F(E(P, K_{t-2}))$$

- 随机函数F实行置换操作防止链合并

原加密链

$$K_0 = SP$$

$$K_1 = E(P, SP)$$

$$K_2 = E(P, K_1)$$

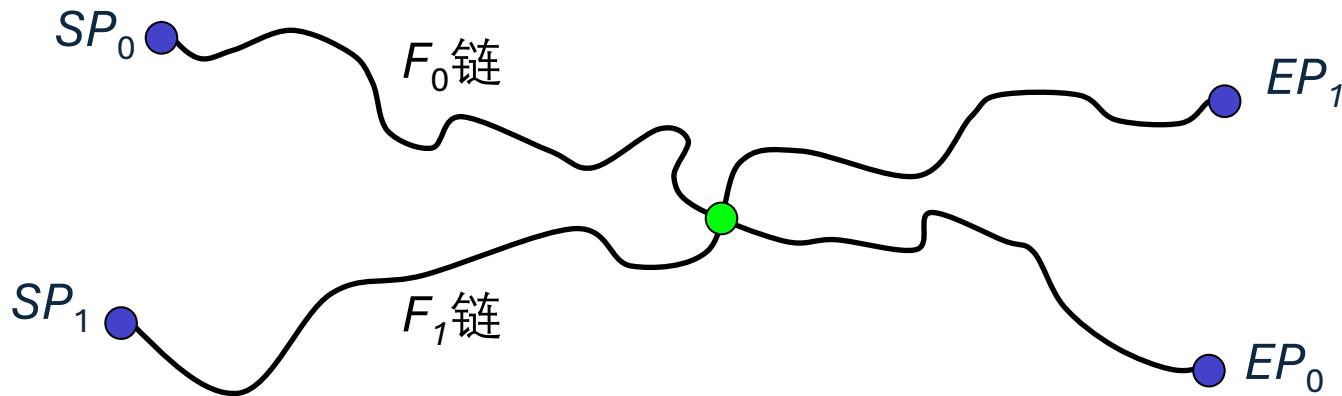
$$K_3 = E(P, K_2)$$

⋮

$$K_{t-1} = EP = E(P, K_{t-2})$$

现实世界的加密链

- 计算链 $F(E(P, K_{i-1}))$, 其中 F 对 bit 进行置换
- 使用不同函数计算出的链可能相交, 但它们**不会**合并。



TMTO成功概率

- 让
 - m = 每个表中链的个数
 - t = 每条链中的长度, 即加密次数
 - r = 表的个数
- 然后 mtr 即为预计算链元素的总数
- 预计算的工作量为 $O(mtr)$
- 每次 TMTO 攻击都需要
 - $O(mr)$ “记忆”和 $O(tr)$ “时间”

TMTO成功概率

- 使用TMTO成功找到一个密钥的概率大约为:

$$P(\text{success}) = 1 - e^{-mtr/2^k}$$

- Hellman给出了mtr的选择建议为:

$$m = t = r = 2^{k/3}$$

- 此时成功概率为0.63

TMTO成功概率估计表

mtr	$P(\text{成功})$
0	0
2^{k-5}	0.03
2^{k-4}	0.06
2^{k-3}	0.12
2^{k-2}	0.22
2^{k-1}	0.39
<u>2^k</u>	<u>0.63</u>
2^{k+1}	0.86
2^{k+2}	0.98
2^{k+3}	0.99
∞	1.00

TMTO攻击

- 针对DES的攻击是可行的。
- 预计算大约需要 2^{56} 个工作量。（密钥长度为56位）
- 每次攻击大约需要
 - 2^{37} “存储” ($2^{2k/3}$)
 - 2^{37} “时间” ($2^{2k/3}$)
- 这种攻击并非建立在DES特有结构上。
- 无需复杂的数学运算！
- 教训： **巧妙的算法可以破解密码！**

Next week

- 访问控制
 - 认证——谁可以去做？
 - 授权——是否被允许做某件事？
- 通过使用密码学来帮助完成认证和授权。

Thank you.